Instituto Tecnológico y de Estudios Superiores de Monterrey



Activity 2.6

Position and speed control loop

By Team 7
Sondre T. Hjemgaard A01833522
Hakon Munch Haraldsson A01832889
Filip Repstad Spanne A01833674

Course
MR2006B

Taught by
Federico Guedea Elizalde

Campus Monterrey Nuevo León

Note on the measurements

The open-loop and closed-loop tests of points 1, 7 and 14 could not be recorded on the car. On the night the tests were planned the battery read $9.4\,\mathrm{V}$ of $12.6\,\mathrm{V}$ (about $3.1\,\mathrm{V}$ per cell, practically empty) and the labs, where it can be charged, were closed until after the deadline. With the professor's agreement the tests were **simulated** for this activity.

To keep the simulated tests as close to the car as possible, a *virtual car* was written (`virtual_car.py` in `matlab/python`). It runs the firmware's own control code (100 ms interrupt, whole encoder counts, the speed filter, the trapezoid and `ControlPID()`) on a plant model, and writes the same telemetry files as the real car. Its parameters are:

- dead zone $p_0 = 50\,\%$ and gain $K = 1.8\,\mathrm{cm/s}$ per %: taken from the feed-forward tuned by hand on the real car until it drove exactly $1\,\mathrm{m}$ (offset 50 % PWM, $0.556\,\%$ per cm/s);
- time constant $\tau = 0.25\,\mathrm{s}$: **assumed**, typical of a small gear-motor car; it was not measured.

Every figure that comes from the virtual car says so, and Section 7 shows how the results change when these assumptions are wrong. The method (and all the scripts) are the same as for real logs: only the data folder changes.

1. Open-loop tests

The car is driven without the controller using the Bluetooth command `P xx` (motor at xx % PWM), with telemetry on (`T 10`: one line every $100\,\mathrm{ms}$ with time, encoder count, distance, speed, motor %). The procedure for each test is `T 10`, $1\,\mathrm{s}$ standing still, `P xx`, $3\,\mathrm{s}$ driving, `X` (brake). Four steps were run: $60\,\%$, $70\,\%$, $80\,\%$ and $90\,\%$.

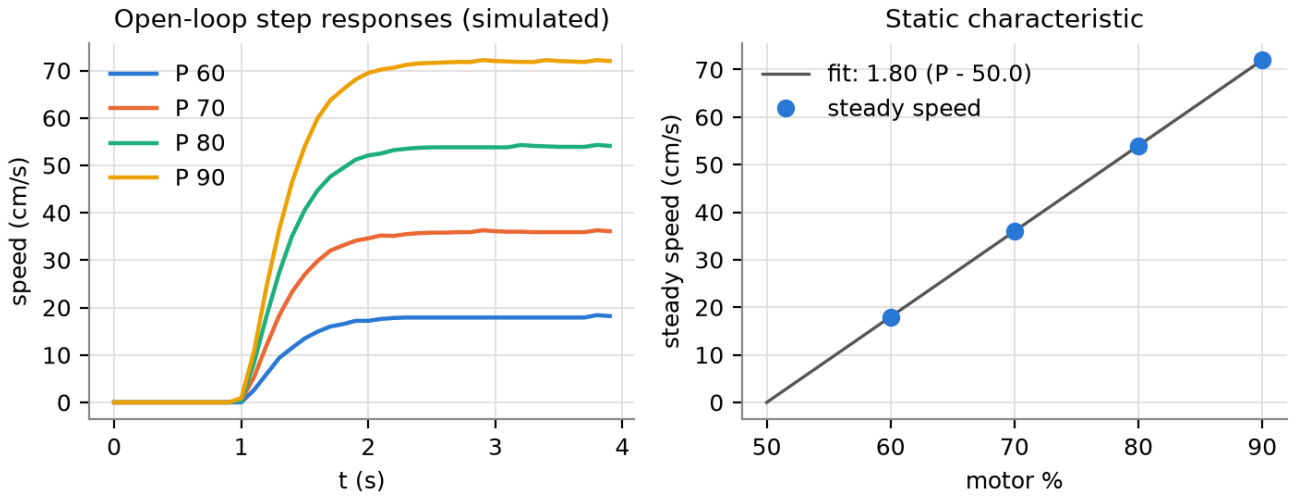


Figure 1: Open-loop step responses of the virtual car (left) and steady-state speed against motor command (right). The speed is the firmware's filtered speed `AverageSpeed`.

The steady-state speed (mean of the last second of each step) is linear in the command above a dead zone, $v_{ss} = k\,(P - p_0)$. A least-squares line through the four points (Figure 1, right) gives $p_0 = 50.0\,\%$ and $k = 1.80\,\mathrm{cm/s}$ per %, the values of the virtual car, which checks the measurement chain. Below p_0 the motor torque does not overcome the friction and the car does not move, so the plant is identified from the input $u_p = P - p_0$.

2. Identification of the model

The four steps are resampled at $T_s = 0.1\,\mathrm{s}$ and merged into one `iddata` set (input $P - p_0$ in %, output speed in cm/s). The System Identification Toolbox (`tfest`) fitted three structures; the same fit was repeated in Python (least squares on the simulated response) to check it.

Table 1: Identified models, speed (cm/s) over $P - p_0$ (%). Fit is the normalized root-mean-square fit of MATLAB's `compare`.

Structure	$G(s)$	Fit
1st order	$\dfrac{1.824}{0.387s + 1}$	94.7 %
1st order + dead time	$\dfrac{1.811\, e^{-0.050s}}{0.329s + 1}$	97.7 %
2nd order	$\dfrac{1.803}{(0.284s + 1)(0.090s + 1)}$	98.8 %

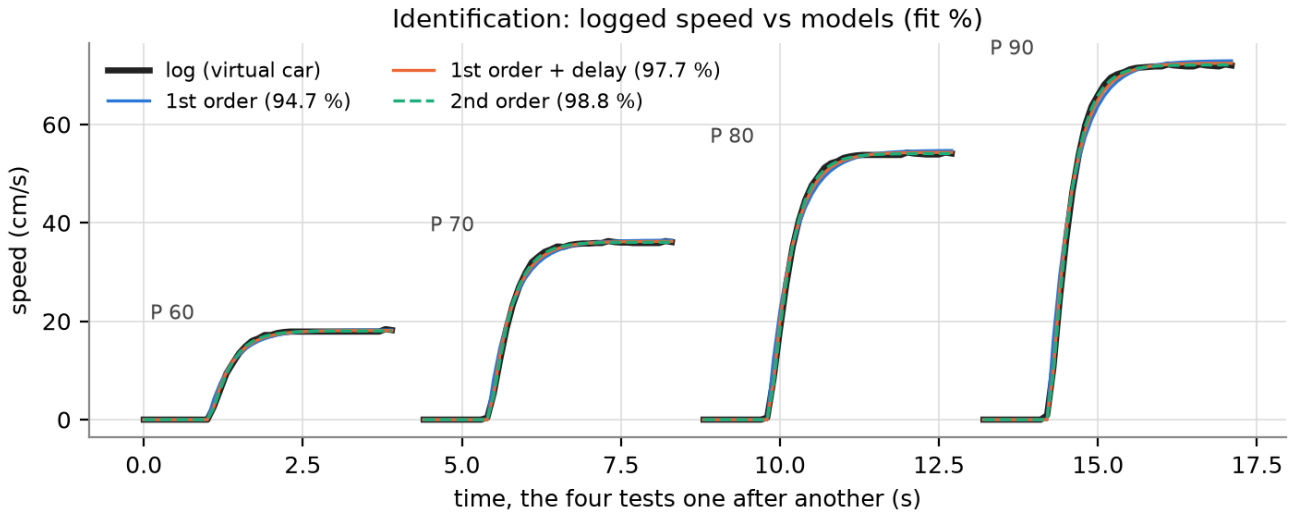


Figure 2: Logged speed of the four steps (virtual car) against the three identified models.

The simplest model is kept unless another one fits more than 3 points better. The 1st order + dead time model is chosen:

$$G(s) = \frac{V(s)}{U_p(s)} = \frac{1.811\, e^{-0.050s}}{0.329s + 1} \quad \left[\frac{\mathrm{cm/s}}{\%}\right], \qquad K = 1.81,\ \tau = 0.33\,\mathrm{s},\ L = 0.05\,\mathrm{s}.$$

Identified on the 60, 80 and 90 % steps only, the same structure predicts the 70 % step with a fit of 97.8 %, so the model is not just following the data it was fitted to.

The identified time constant (0.33 s) is larger than the plant's (0.25 s) and a dead time appears. Both come from the speed measurement, not from the motor: the firmware computes the speed from the counts of the last 100 ms (an average, so on average half a period late) and then filters it with $\bar{v}_k = 0.5\,\bar{v}_{k-1} + 0.5\,v_k$. The model therefore describes what the controller sees, which is what the tuning needs.

Screenshot to add: `figures/matlab_ident.png`
System Identification app (`systemIdentification`, import `dat`) or the `compare` plot from `step1_identify`

Figure 3: Identification in MATLAB (`step1_identify.m`).

3. Closed loop in Simulink: sensors, actuators and target

The car is controlled in position. The target is a distance D in a total time T with a trapezoidal speed profile: acceleration time t_a and deceleration time t_d set by the user (Bluetooth commands `tt`, `ta`, `td`). The area under the profile is the distance, which gives the nominal speed:

$$D = \frac{S_n t_a}{2} + S_n(T - t_a - t_d) + \frac{S_n t_d}{2} \quad \Rightarrow \quad S_n = \frac{D}{T - (t_a + t_d)/2}.$$

For the test run, $D = 100\,\mathrm{cm}$, $T = 8\,\mathrm{s}$, $t_a = t_d = 2\,\mathrm{s}$, so $S_n = 100/6 = 16.7\,\mathrm{cm/s}$. The speed profile `SP_Vel` is used as feed-forward and its integral `Pos_Ref` as the position reference.

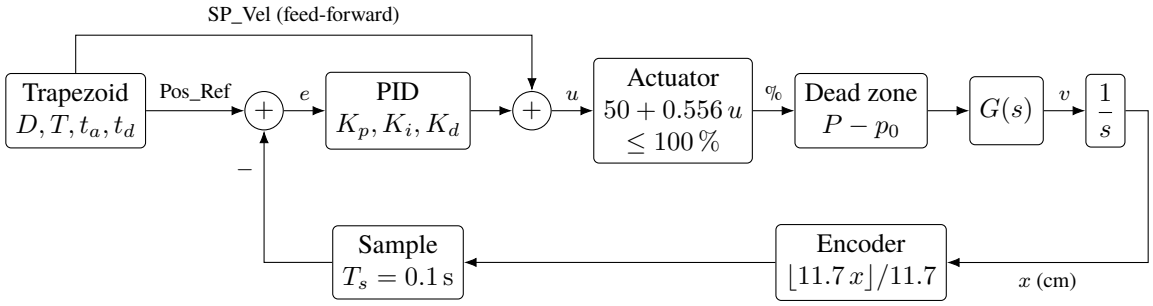


Figure 4: Position loop as implemented on the car and in Simulink. Units: position in cm, speed and controller output u in cm/s, actuator output in % PWM.

The models of the sensors and the actuator are the ones in the firmware:

- **Actuator** (H-bridge + motor command): the controller output u in cm/s becomes PWM through the feed-forward line found on the car, $P = 50 + 0.556\,u$, limited to 100 %; the plant then sees $P - p_0$.
- **Plant**: $G(s)$ from Section 2, speed in cm/s, integrated to position.
- **Sensor** (encoder): whole counts only, 1170 counts per metre ($c = 11.7$ counts/cm), read every T_s; this is a quantizer plus a sampler.

The Simulink model (`build_simulink_model.m`) contains two copies of this loop, one with a continuous PID and one with the discrete PID, sampler and quantizer, driven by the same reference.

Screenshot to add: `figures/simulink_model.png`
Diagram of `car_position_loop.slx` (built by `build_simulink_model.m`)

Figure 5: Simulink model of the continuous and discrete position loops.

4. Continuous PID simulation

The PID sees the plant from its output u (cm/s) to the position (cm). The constant offsets of the actuator only move the operating point, so the linear plant is

$$P_c(s) = 0.556\, G(s)\, \frac{1}{s} = \frac{1.007\, e^{-0.050 s}}{s\,(0.329 s + 1)}.$$

The loop gain is almost exactly 1 because the feed-forward slope (0.556) is the inverse of the plant gain (1.81): the feed-forward alone already drives the car at the right speed, and the PID only has to correct the error. Figure 6 shows the timed run with the continuous PI tuned in Section 6 (blue).

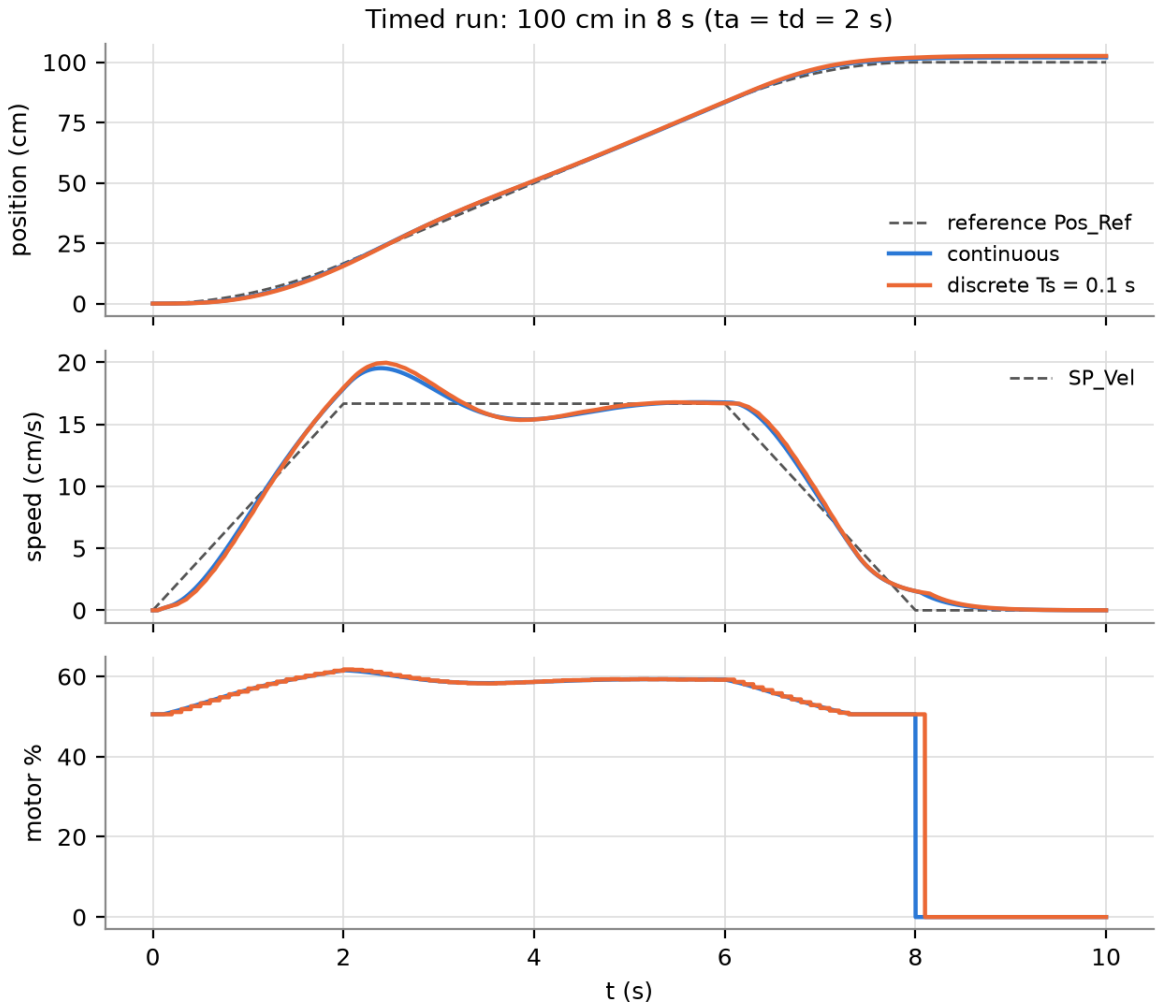


Figure 6: Simulated timed run, $100\,\mathrm{cm}$ in $8\,\mathrm{s}$: continuous PI ($K_p = 2.04$, $K_i = 1.25$) and discrete PI at $T_s = 0.1\,\mathrm{s}$ ($K_p = 2.06$, $K_i = 0.90$).

5. Discretization and sampling time

The discrete loop samples the encoder and updates the motor every T_s, with the plant seen through a zero-order hold, $P_d(z) = \mathcal{Z}\{\mathrm{ZOH} \cdot P_c(s)\}$ (`c2d(Pc, Ts, 'zoh')`). The PID is discretized exactly as `ControlPID()` computes it, backward Euler for both the integral and the derivative:

$$C(z) = K_p + K_i \frac{T_s\, z}{z-1} + K_d \frac{z-1}{T_s\, z}.$$

Three arguments fix $T_s = 0.1\,\mathrm{s}$:

- **Bandwidth.** The tuned continuous loop has a closed-loop bandwidth of $3.26\,\mathrm{rad/s}$. Sampling at $\omega_s = 2\pi/T_s = 62.8\,\mathrm{rad/s}$ is 19 times faster, inside the usual 10–30 times.
- **Encoder resolution.** At the cruise speed of $16.7\,\mathrm{cm/s}$ the car advances $16.7 \cdot 11.7 \cdot T_s = 19.5$ counts per period at $T_s = 0.1\,\mathrm{s}$, so the speed is known to one count in 20 (5 %). At $0.02\,\mathrm{s}$ it would be 4 counts and a resolution of 26 %.

- **Simulation** (Table 2, Figure 7): the same gains at different T_s. Going from $0.02\,\mathrm{s}$ to $0.1\,\mathrm{s}$ adds only $0.14\,\mathrm{cm}$ of RMS error; above $0.2\,\mathrm{s}$ it grows quickly.

Table 2: Tracking error of the timed run for different sampling times (discrete PI, $K_p = 2.06$, $K_i = 0.90$). Final error: negative means the car stops past $100\,\mathrm{cm}$.

T_s (s)	RMS error (cm)	Max error (cm)	Final error (cm)
continuous	0.97	1.53	-2.0
0.02	1.04	1.74	-2.3
0.05	1.09	1.82	-2.4
0.10	**1.18**	**1.97**	$\mathbf{-2.6}$
0.20	1.37	2.28	-3.0
0.40	1.89	3.02	-3.9

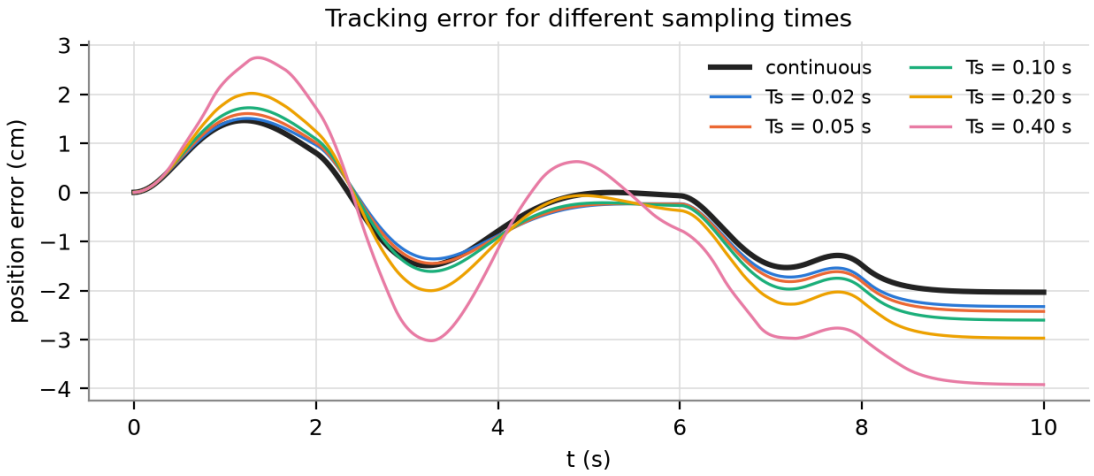


Figure 7: Position error during the run for different sampling times.

6. Tuning of the discrete PID

The PI gains are found with `pidtune` on $P_d(z)$ for a target phase margin and crossover frequency. For a PI, the crossover ω_c and the phase margin fix both gains: at ω_c, $C(e^{j\omega_c T_s})\,P_d(e^{j\omega_c T_s})$ must have magnitude 1 and phase $-180^\circ + \mathrm{PM}$. The phase margin was chosen by simulating the timed run for each target, both on the identified model and with the dead zone off by $\pm 5\,\%$ and the gain off by $\pm 20\,\%$ (worst case of nine combinations). The crossover is the one with the smallest worst-case error.

Table 3: Discrete PI for different phase-margin targets. RMS error of the timed run, nominal and worst of the nine mismatch cases.

Design	ω_c (rad/s)	K_p	K_i	GM (dB)	RMS nominal (cm)	RMS worst (cm)
PM 30°	2.10	2.41	1.23	10.9	1.01	2.87
PM 35°	**1.85**	**2.06**	**0.90**	**12.5**	**1.18**	**3.14**
PM 45°	1.55	1.69	0.42	15.0	1.43	4.35
PM 60°	1.20	1.28	0.04	18.2	1.65	8.80
Hand-tuned on the car (PM 19°)	2.90	3.80	2.50	7.3	0.59	2.66

The usual target of 60° does not work here: the plant already has -90° from the integrator plus the lag of τ, the dead time and the sampling, so 60° is only reachable at a low crossover, where the integral gain drops to almost zero and the worst-case error is three times larger. 35° is the compromise, with a gain margin of $12.5\,\mathrm{dB}$:

$$\text{continuous: } K_p = 2.04,\ K_i = 1.25; \qquad \text{discrete } (T_s = 0.1\,\mathrm{s}):\ K_p = 2.06,\ K_i = 0.90,\ K_d = 0.$$

The discrete K_i is lower than the continuous one because the sample and hold adds about $\omega_c T_s/2 = 5^\circ$ of lag at the crossover, which the PI makes up for with less integral phase lag. K_d was left at zero: the timed run has no sudden reference changes and the derivative of a 20-count signal is noisy.

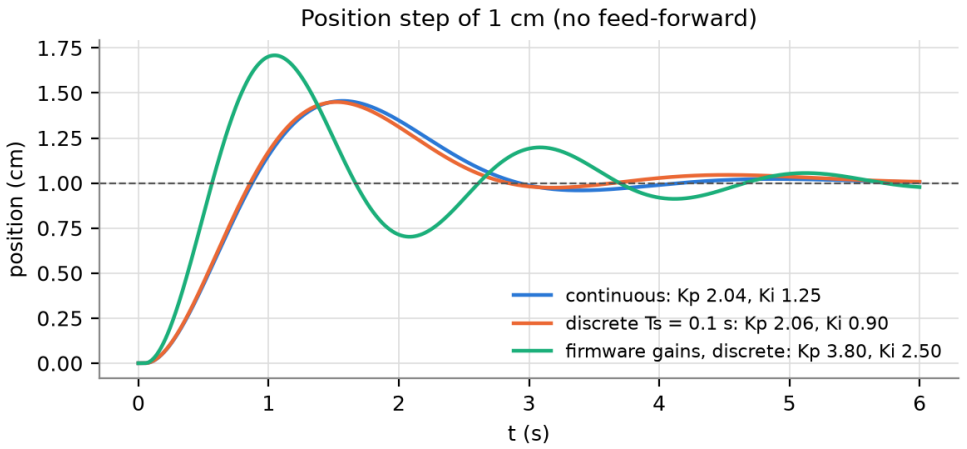


Figure 8: Response to a $1\,\mathrm{cm}$ position step without feed-forward (the worst case for the PID alone).

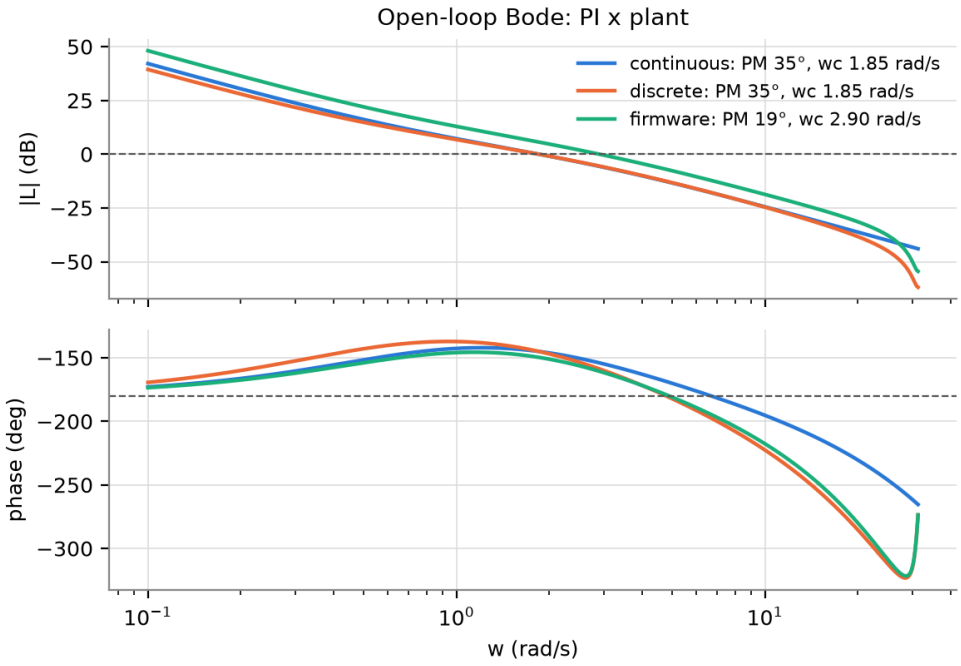


Figure 9: Open-loop Bode diagram, $C \cdot P$, with phase margins and crossovers.

7. Validation of the simulation

The simulation was checked in three ways.

Model. The model predicts a step that was not used to fit it with 97.8 % fit (Section 2).

Closed loop. A timed run (`tt 8`, `ta 2`, `td 2`, `d1`, gains $K_p = 3.8$, $K_i = 2.5$, $K_d = 0.05$) was logged from the virtual car and compared with the discrete simulation of Section 5 (`step4_validate.m`), Figure 10. The two positions differ by $0.55\,\mathrm{cm}$ RMS; the car stops at $100.8\,\mathrm{cm}$ and the simulation at $101.0\,\mathrm{cm}$. The speed in the simulation leads the logged `AverageSpeed` slightly, because the log contains the filter delay that the model represents with a dead time.

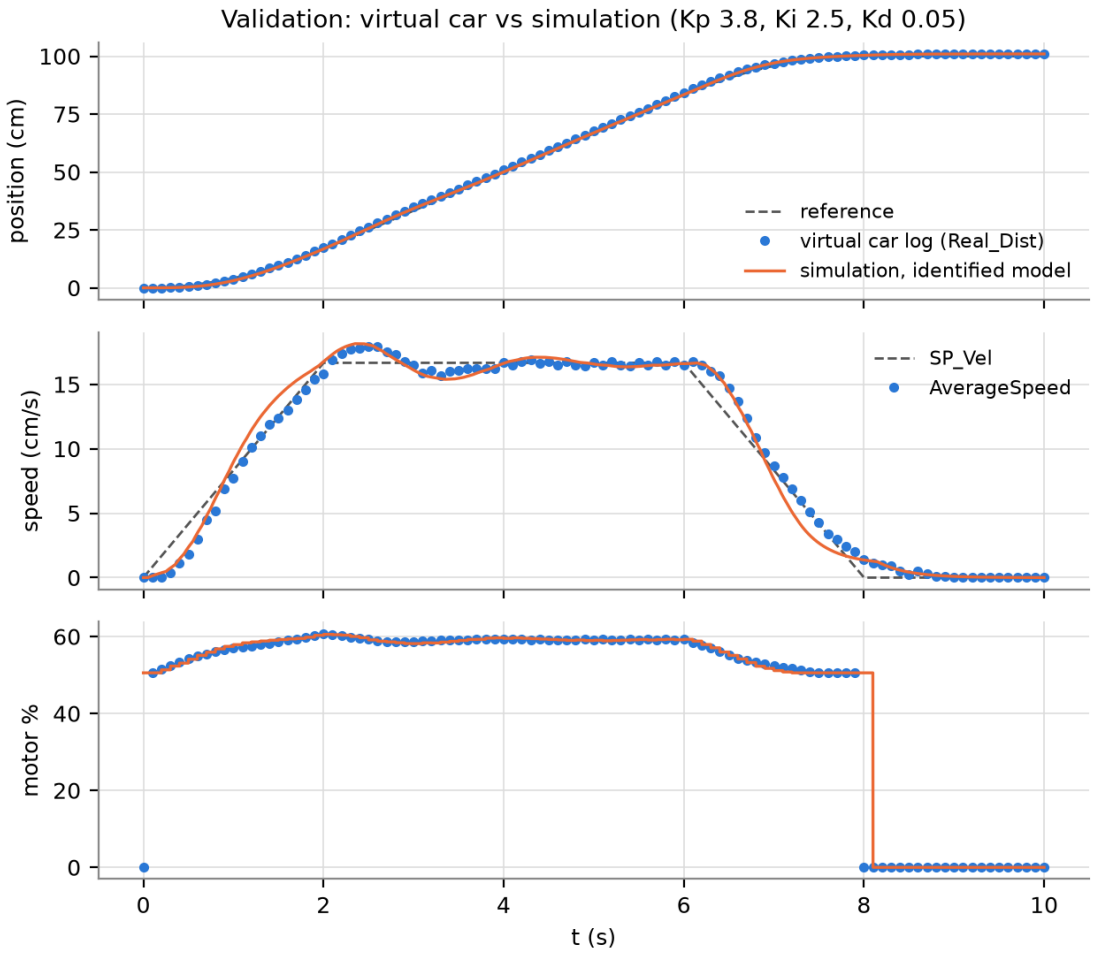


Figure 10: Validation: logged run of the virtual car against the discrete simulation with the identified model.

Robustness to the assumptions. The virtual car's τ was assumed and its p_0 and K were derived from the feed-forward, so the timed run was repeated on the virtual car with each parameter changed (Table 4).

Table 4: Timed run of $100\,\mathrm{cm}$ on the virtual car when its parameters differ from the assumptions: final position and RMS tracking error.

	Tuned PI (2.06, 0.90)		Hand-tuned (3.8, 2.5, 0.05)	
Virtual car	Final (cm)	RMS (cm)	Final (cm)	RMS (cm)
As assumed	101.7	0.86	100.8	0.43
$\tau = 0.10\,\mathrm{s}$	100.9	0.46	100.5	0.23
$\tau = 0.50\,\mathrm{s}$	104.8	1.74	102.9	0.90
$p_0 = 45\,\%$	110.9	2.10	110.3	1.47
$p_0 = 55\,\%$	101.2	2.18	100.4	1.05
$K - 20\,\%$	103.1	1.36	101.5	0.69
$K + 20\,\%$	100.8	0.84	100.5	0.40

Both gain sets stay stable in every case. The one large error, $p_0 = 45\,\%$, is a limitation of the firmware rather

than of the gains: during a run the command is never below $1\,\mathrm{cm/s}$, i.e. 50.6 % PWM, and a car that already moves at 45 % cannot slow down enough during the deceleration. On the real car this would show as an overshoot of the target; the fix would be to allow a lower command, or to brake, at the end of the run.

8. Sensors and actuators

Table 5: Sensors and actuators used for the control and status of the car.

Device	Role	Interface
DC gear motor + L298N H-bridge	Actuator: drives the car	PWM (TIM1) + 2 GPIO
Quadrature hall encoder on the motor	Sensor: position and speed	Encoder mode (TIM5)
HC-SR04 ultrasonic sensor	Sensor: obstacle distance	GPIO trigger + EXTI echo
Bluetooth module (HC-05 type)	Commands and telemetry	USART1, $9600\,\mathrm{baud}$
SSD1306 OLED display 128×64	Internal status	I2C1
Steering servo	Actuator: steering (not in the loop)	PWM (TIM2)
Battery divider 10k/3.3k	Battery voltage	ADC1

9. Connections

Figure 11 shows how the devices are connected to the NUCLEO-F446RE. The HC-SR04 echo is a $5\,\mathrm{V}$ signal, so it passes through a 1k/2k divider to $3.3\,\mathrm{V}$ before PA7. The board is powered from the car (jumper JP5 on E5V); the USB cable is only for programming and the virtual COM port.

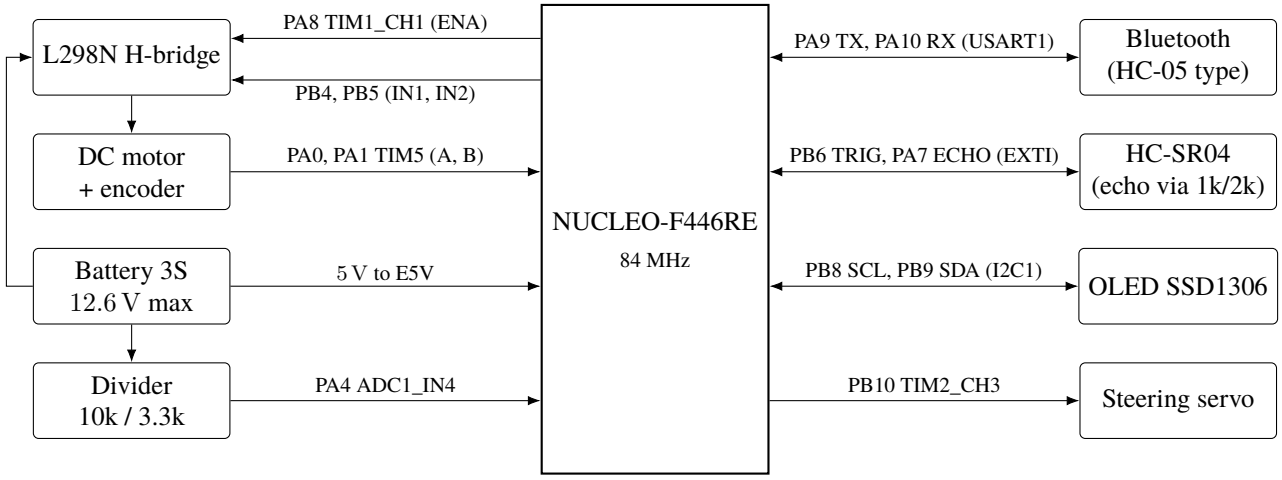


Figure 11: Connection diagram. Arrows point from the source of the signal; the battery feeds the H-bridge directly and the board through the car's $5\,\mathrm{V}$ supply (jumper JP5 on E5V).

Screenshot to add: `figures/cubemx_pinout.png`
CubeMX *Pinout view* of `stm32-car.ioc`

Figure 12: Pin assignment in STM32CubeMX.

10. Sampling period and PWM: timer configuration

The system clock is $84\,\mathrm{MHz}$ (HSI $16\,\mathrm{MHz}$ through the PLL, $N = 336$, $P = 4$). APB1 runs at $42\,\mathrm{MHz}$; because its prescaler is not 1 the timer clock is doubled, so all timers count at $f_{\mathrm{clk}} = 84\,\mathrm{MHz}$. For a timer with prescaler register PSC and auto-reload register ARR (both counted from zero, hence the $+1$):

$$f = \frac{f_{\mathrm{clk}}}{(\mathrm{PSC}+1)(\mathrm{ARR}+1)}.$$

Control interrupt, TIM3. Internal clock, no channels, update interrupt enabled in the NVIC. With PSC = 839 and ARR = 9999:

$$f_{\mathrm{int}} = \frac{84\cdot 10^6}{840\cdot 10\,000} = 10\,\mathrm{Hz}, \qquad T_s = \frac{1}{f_{\mathrm{int}}} = 100\,\mathrm{ms}.$$

Its counter runs at $84\cdot 10^6/840 = 100\,\mathrm{kHz}$ (one tick every $10\,\mu\mathrm{s}$), which is also used to time the ultrasonic echo (Section 15). The timer is started with `HAL_TIM_Base_Start_IT()`, and its period-elapsed callback calls `drive_update()` every period.

Motor PWM, TIM1 channel 1. PSC = 0 and ARR = 16 799:

$$f_{\mathrm{PWM}} = \frac{84\cdot 10^6}{1\cdot 16\,800} = 5\,\mathrm{kHz}, \qquad \mathrm{CCR} = \frac{P}{100}\,(\mathrm{ARR}+1),$$

so a command of $P = 59.3\,\%$ (the cruise of the test run) writes $\mathrm{CCR} = 0.593\cdot 16\,800 = 9962$. $5\,\mathrm{kHz}$ is well above the motor's electrical time constant, so the current (and torque) follows the average voltage.

Encoder, TIM5. Encoder mode TI1 and TI2 (both edges of A and B, $\times 4$ decoding), input filter 6, 32-bit counter (ARR = $2^{32}-1$), so it never wraps during a run and reverse motion gives negative counts.

Screenshot to add: `figures/cubemx_tim3.png`
CubeMX TIM3 parameter settings (PSC 839, ARR 9999)

Screenshot to add: `figures/cubemx_tim1.png`
CubeMX TIM1 parameter settings (PSC 0, ARR 16799, PWM CH1)

Figure 13: Timer configuration in STM32CubeMX.

Screenshot to add: `figures/cubemx_clock.png`
CubeMX *Clock Configuration* (84 MHz, APB1 timer clock 84 MHz)

Figure 14: Clock tree.

11. Selection of the PID values

The simulations give two candidate sets (Table 3 and Table 4):

- the systematic design, $K_p = 2.06$, $K_i = 0.90$, $K_d = 0$: PM $35°$, GM $12.5\,\mathrm{dB}$;
- the gains tuned by hand on the car while getting it to drive exactly $1\,\mathrm{m}$, $K_p = 3.8$, $K_i = 2.5$, $K_d = 0.05$: PM $19°$, GM $7.3\,\mathrm{dB}$.

The hand-tuned set tracks the trapezoid about twice as well in every case of Table 4, because the feed-forward makes the motion and the PID only corrects small errors, so a higher gain helps without exciting oscillation. Its margins are small, however, and a step without feed-forward overshoots by 70 % (Figure 8).

Selected: $K_p = 3.8$, $K_i = 2.5$, $K_d = 0.05$ remain the firmware defaults, since they also worked on the real car and give the best simulated tracking with a gain margin above $6\,\mathrm{dB}$. The tuned set (`kp 2.06`, `ki 0.9`, `kd 0` over Bluetooth) is the fallback if the car oscillates, for example with a fully charged battery, which raises the plant gain.

12. PID on the microcontroller

Everything runs in `drive_update()`, called from the TIM3 interrupt every $T_s = 0.1\,\mathrm{s}$, so the step of the integral and the derivative is always the same.

Distance. The encoder count C_k is converted with the calibration constant c (counts per cm), found by driving a measured metre and adjustable over Bluetooth (`M`):

$$x_k = \frac{|C_k|}{c}, \qquad c = \frac{1170 \text{ counts/m}}{100} = 11.7 \text{ counts/cm}.$$

Speed. From the counts of the last period, then a first-order filter:

$$v_k = \frac{C_k - C_{k-1}}{c\,T_s}\ [\mathrm{cm/s}], \qquad \bar{v}_k = 0.5\,\bar{v}_{k-1} + 0.5\,v_k.$$

One count in one period is $1/(11.7 \cdot 0.1) = 0.85\,\mathrm{cm/s}$, the speed resolution.

Reference. A state machine over the elapsed time t_k gives the profile, and the reference position is its running sum:

$$\text{SP_Vel}_k = \begin{cases} S_n\,t_k/t_a & t_k < t_a \\ S_n & t_a \le t_k < T - t_d \\ S_n\,(T - t_k)/t_d & T - t_d \le t_k < T \end{cases}, \qquad \text{Pos_Ref}_k = \text{Pos_Ref}_{k-1} + \text{SP_Vel}_k\,T_s.$$

Control law.

$$e_k = \text{Pos_Ref}_k - x_k, \qquad I_k = \text{sat}_{\pm 25}(I_{k-1} + e_k T_s)\,,$$
$$u_k = \text{SP_Vel}_k + K_p e_k + K_i I_k + K_d \frac{e_k - e_{k-1}}{T_s}, \qquad P_k = \min(50 + 0.556\ \max(u_k, 1),\ 100)\,\%.$$

The integral is clamped to $\pm 25\,\text{cm s}$ (anti-windup). When $t_k \geq T$ the motor is switched off and the car coasts to a stop.

```
float ControlPID(float setpoint, float input, float dt) {
  error = setpoint - input;                        /* cm */
  ErrorAcum = ErrorAcum + (error * dt);            /* integral, backward Euler */
  if (ErrorAcum > 25.0f)  ErrorAcum = 25.0f;       /* anti-windup */
  if (ErrorAcum < -25.0f) ErrorAcum = -25.0f;
  ErrorRate = (dt > 0.0f) ? (error - lastError) / dt : 0.0f;
  float output = (Kp * error) + (Ki * ErrorAcum) + (Kd * ErrorRate);
  lastError = error;
  return output;                                   /* cm/s, added to SP_Vel */
}
```

Listing 1: The PID, `ControlPID()` in `car_drive.c`.

13. OLED status display

A 128×64 SSD1306 OLED on I2C1 (PB8/PB9; the firmware finds it at address 0x3C or 0x3D) is redrawn from the main loop, outside the interrupt, so the control timing is not affected. It shows: the battery voltage and the obstacle distance (top line); the state of the run (`IDLE`, `ACCEL`, `CRUISE`, `DECEL`, `OBSTACLE`, or the last Bluetooth byte received) and the target distance; the distance driven in large digits and the motor %; the measured speed `AverageSpeed` and the setpoint `SP_Vel`; and a bar with the motor command, forward to the right. The battery reading is not calibrated yet: it reads low ($5.9\,\text{V}$ for a $9.4\,\text{V}$ pack), which points to a wiring or divider fault on PA4.

14. Verification against MATLAB

The variables of the loop (`Real_Dist`, `Pos_Ref`, `SP_Vel`, `AverageSpeed`, `Kp`, `Ki`, `Kd`) are global and `volatile` so STM32CubeMonitor can plot them and change the gains live, and the same values are sent as telemetry (`T 10`) over Bluetooth and USB. Because the car could not be run with a charged battery, the comparison with MATLAB was made with the telemetry of the virtual car instead of a CubeMonitor recording of the real car (Section 7, Figure 10): position within $0.55\,\text{cm}$ RMS, final position $100.8\,\text{cm}$ against $101.0\,\text{cm}$ simulated. With a charged battery the same comparison is made by logging a `d1` run to `matlab/data/` and running `step4_validate.m`.

15. Obstacle stop

The HC-SR04 is triggered at the end of every control step (a $10\,\mu\text{s}$ pulse on PB6). The echo on PA7 raises an interrupt on both edges, and the pulse width is measured with the TIM3 counter ($10\,\mu\text{s}$ per tick). Sound travels the distance twice at about $343\,\text{m/s}$, so

$$d\,[\text{cm}] = \frac{t_{\text{echo}}\,[\mu\text{s}] \cdot 0.0343}{2} \approx \frac{t_{\text{echo}}}{58}.$$

Readings below $2\,\mathrm{cm}$ are rejected as noise. When two readings in a row are at most $30\,\mathrm{cm}$, the run is paused: the motor brakes, and the profile clock and `Pos_Ref` are frozen, so the PID does not build up error while the car waits. When the obstacle is further than $35\,\mathrm{cm}$ (the $5\,\mathrm{cm}$ hysteresis avoids chattering at the threshold), the run continues from the same point of the profile and the OLED stops showing `OBSTACLE`. In this version the check is only active during a run (`d` command), not in manual driving.

16. Bluetooth control

The Bluetooth module (HC-05 type, serial port profile) is on USART1 at $9600\,\mathrm{baud}$; each received byte raises an interrupt and a full line is handled in the main loop. The same commands work from the USB virtual COM port. Replies and telemetry go to both.

Table 6: Bluetooth commands used for the runs and the tuning.

Command	Action	Example
`d`x	Start a run of x metres	`d1`, `d1.5`
`tt`, `ta`, `td`	Total, acceleration and deceleration time (s)	`tt 8`
`kp`, `ki`, `kd`	PID gains	`kp 3.8`
`P` p	Open-loop motor command in %	`P 70`
`X`, `S`	Stop with brake / stop	
`T` f	Telemetry at f Hz, `T 0` off	`T 10`
`M` c	Encoder counts per metre	`M 1170`
`G`	Print gains, times, sonar, battery	
`f`, `b`, `l`, `c`, `r`	Manual drive and steering	

The Bluetooth link at $9600\,\mathrm{baud}$ needs about $1.04\,\mathrm{ms}$ per character. A first version sent each line with a fixed $10\,\mathrm{ms}$ timeout, which cut every reply after about ten characters; the timeout now scales with the line length. For the same reason telemetry is sent at $10\,\mathrm{Hz}$: a line of about 45 characters takes $47\,\mathrm{ms}$.

AI audit

AI was used to write and restructure most of the firmware from our earlier code (control interrupt, trapezoid, PID, obstacle stop, OLED and Bluetooth commands), to write the MATLAB and Python scripts, the virtual car used for the simulated tests, and to draft this report and its LaTeX formatting. The wiring, the hardware bring-up, the hand tuning of the feed-forward and the gains on the car, and the videos were done by the team, and the decision to simulate the tests was agreed with the professor because the battery could not be charged. The simulated data are marked as such throughout; we checked the results against the firmware constants and the earlier run on the car, and the choices of sampling time and gains are ours.